
PRACTICAL WEEK 4

1. Write a C program where a parent process creates multiple child processes and synchronizes their completion using wait () and waitpid (). Compare the behavior of both functions.

Create a file:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano practical4.1.c
```

Write the code:

```
GNU nano 8.7.1                                pract
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t children[3];

    printf("Parent process started. PID = %d\n", getpid());

    // Create 3 child processes
    for (int i = 0; i < 3; i++)
    {
        children[i] = fork();

        if (children[i] < 0)
        {
            perror("fork failed");
            exit(EXIT_FAILURE);
        }

        if (children[i] == 0)
        {
            printf("Child %d started. PID = %d\n", i + 1, getpid());

            sleep(2 + i);

            printf("Child %d finished. PID = %d\n", i + 1, getpid());

            exit(10 + i);
        }
    }

    printf("\nParent is waiting for children...\n");

    // wait() waits for any one child
    pid_t pid = wait(NULL);

    printf("wait(): Child with PID %d has completed.\n", pid);
}
```

```

    for (int i = 0; i < 3; i++)
    {
        int status;

        pid = waitpid(children[i], &status, 0);

        if (pid > 0)
        {
            if (WIFEXITED(status))
            {
                printf("waitpid(): Child PID %d completed with exit status %d.\n",
                    pid, WEXITSTATUS(status));
            }
        }
    }

    printf("All child processes have completed.\n");

    return 0;
}

```

Run the code:

```

vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano practical4.1.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ gcc practical4.1.c -o practical4.1
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ |

```

```

vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano practical4.1.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ gcc practical4.1.c -o practical4.1
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./practical4.1|

```

OUTPUT:

```

vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano practical4.1.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ gcc practical4.1.c -o practical4.1
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./practical4.1
Parent process started. PID = 790
Child 1 started. PID = 791
Child 2 started. PID = 792

Parent is waiting for children...
Child 3 started. PID = 793
Child 1 finished. PID = 791
wait(): Child with PID 791 has completed.
Child 2 finished. PID = 792
waitpid(): Child PID 792 completed with exit status 11.
Child 3 finished. PID = 793
waitpid(): Child PID 793 completed with exit status 12.
All child processes have completed.
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ |

```

2. Create a scenario where a child process becomes a zombie process.
Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

Create a file:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano practical4.2.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ |
```

Write the code:

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        exit(EXIT_FAILURE);
    }

    if (pid == 0)
    {
        printf("Child process started. PID = %d\n", getpid());
        printf("Child process is exiting...\n");

        exit(0);
    }
    else
    {
        printf("Parent process started. PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);
        printf("Parent is sleeping for 30 seconds...\n");

        sleep(30);

        printf("Parent exiting.\n");
    }

    return 0;
}
```

Run the code:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano practical4.2.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ gcc practical4.2.c -o practical4.2
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./practical4.2
```

OUTPUT:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./practical4.2
Parent process started. PID = 977
Child PID = 978
Parent is sleeping for 30 seconds...
Child process started. PID = 978
Child process is exiting...
ps -el | grep Z
Parent exiting.
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ps -el | grep Z
F S  UID      PID     PPID  C PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ |
```

Now Eliminate the zombie by modifying the program so the parent calls waitpid ().

Now create a file:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano practical_fixed.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ |
```

Enter the code:

```

GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;
    int status;

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        exit(EXIT_FAILURE);
    }

    if (pid == 0)
    {
        printf("Child process started. PID = %d\n", getpid());
        printf("Child process is exiting...\n");

        exit(0);
    }
    else
    {
        printf("Parent process started. PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);

        waitpid(pid, &status, 0);

        if (WIFEXITED(status))
        {
            printf("Child %d terminated normally.\n", pid);
            printf("Exit status = %d\n", WEXITSTATUS(status));
        }

        printf("No zombie process exists.\n");
    }

    return 0;
}

```

Run the code:

```

vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano practical_fixed.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ gcc practical_fixed.c -o practical_fixed
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./practical_fixed

```

OUTPUT:

```

vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano practical_fixed.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ gcc practical_fixed.c -o practical_fixed
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./practical_fixed
Parent process started. PID = 1091
Child PID = 1092
Child process started. PID = 1092
Child process is exiting...
Child 1092 terminated normally.
Exit status = 0
No zombie process exists.
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ |

```